SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 2

Scenario-Based Questions

COURSE NAME: Deep Learning **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO 2: Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches. (BTL-4)

CO Weightage: 15%

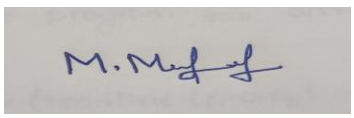
Assessment Tool Weightage: 40%

Duration: 90 minutes

Marks: 25

Code of Conduct:

I M.MEGHANADH/192325026(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Scenario-Based Questions

CO-AT2

QUESTIONS: 05 **Total Marks:25**

1. Unsupervised Training of Neural Networks, Auto Encoders

Scenario: Hospital wants to identify abnormal ECG patterns from thousands of unlabeled records.

Recommended technique

Denoising / Variational Autoencoder (AE/VAE) trained in an unsupervised manner for anomaly detection on ECG time-series.

Justification

- The data are unlabeled and mostly normal, with rare abnormal patterns → ideal for unsupervised representation learning and anomaly detection.acm+2
- Autoencoders learn a compact latent representation of normal ECGs by trying to reconstruct the input; abnormal ECGs will have high reconstruction error, flagging them as anomalies.
- Literature shows autoencoders (including LSTM/denoising/variational AEs) outperform many baselines for ECG anomaly detection and disease profiling from unlabeled ECGs.

Workflow

1. Preprocessing
    - Clean raw ECG signals (filter noise, normalize amplitude/time).
    - Segment into fixed-length windows (e.g., one heartbeat or fixed time window).
2. Model architecture
    - Encoder: 1D CNN and/or LSTM layers to compress each ECG segment into a low-dimensional latent vector.
    - Decoder: Symmetric 1D CNN/LSTM to reconstruct the original segment from the latent code.
    - Optionally use a denoising autoencoder (input corrupted slightly, target is clean signal) to improve robustness.
3. Training (unsupervised)
    - Train on all (or predominantly normal) ECG segments to minimize reconstruction loss (e.g., MSE).
    - No labels needed; loss = difference between input and reconstructed output.
4. Anomaly scoring
    - For each new ECG segment, compute reconstruction error.
    - Define a threshold on reconstruction error (e.g., using percentile of errors on a "normal" validation set, or methods like Kapur's thresholding).
    - Segments with error > threshold → abnormal; others → normal.
5. Post-processing & clinical use
    - Aggregate segment-level scores to patient-level risk.
    - Present flagged ECGs to clinicians for review.

This directly matches the syllabus topics: unsupervised training, autoencoders, and representation learning for anomaly detection.

2. Auto Encoders, Representation Learning

Scenario: Retail company wants to reduce dimensionality of customer purchase data before segmentation.

How an autoencoder helps

An autoencoder can compress high-dimensional purchase vectors (many products, time periods, features) into a low-dimensional latent space that preserves the essential structure of customer behavior, then clustering is performed on this compact representation.

Workflow

1. Data preparation
    - Build a feature matrix: rows = customers, columns = purchase features (e.g., counts/amounts per product category, recency, frequency, monetary value, etc.).
    - Normalize/scale features (e.g., MinMax or standardization).

2. Autoencoder design
    - Input layer: dimension = number of original features.
    - Encoder: several fully connected layers shrinking to a bottleneck (e.g., 2–16 dimensions).
    - Decoder: symmetric layers expanding back to original dimension.
    - Loss: reconstruction error (MSE or similar).

3. Training
    - Train the autoencoder in an unsupervised way on all customers' purchase vectors.
    - The network learns to reconstruct inputs via the bottleneck; the bottleneck activations become the learned representations.

4. Dimensionality reduction
    - For each customer, take the encoder's output at the bottleneck as their new feature vector (e.g., 8D or 2D instead of hundreds of dimensions).
    - This latent space captures non-linear relationships and removes noise/redundancy better than linear methods like PCA in many cases.

5. Customer segmentation
    - Run a clustering algorithm (K-Means, Mini-Batch K-Means, DBSCAN, etc.) on the latent vectors.
    - Use silhouette score / elbow method to choose number of clusters.
    - Analyze each cluster's average purchase behavior to define segments (e.g., "high-value frequent buyers", "discount seekers", "occasional big spenders").

Literature on retail segmentation shows autoencoder + clustering yields more coherent and actionable customer groups than using raw high-dimensional data.

3. Width and Depth of Neural Networks, Activation Functions

Scenario: Facial recognition system performs poorly with a shallow neural network.

Recommended architectural changes

1. Increase depth (more layers)
    - Move from a shallow network to a deep convolutional neural network (CNN) with multiple convolution + pooling blocks followed by fully connected layers.
    - Depth allows hierarchical feature learning: edges → textures → parts → faces.link.
2. Increase width appropriately
    - Within each layer, use enough filters/units to capture variability (e.g., 32, 64, 128, 256 filters in successive conv layers).
    - Avoid making every layer extremely wide; instead, grow width gradually with depth.
3. Use suitable activation functions
    - Replace sigmoid/tanh in hidden layers with ReLU or variants (Leaky ReLU, Parametric ReLU).
        - ReLU: $f(x) = \max(0, x)$reduces vanishing gradient and speeds up training.
    - Use softmax only at the output for multi-class face identity classification.
4. Add regularization and normalization
    - Batch Normalization after conv/linear layers to stabilize training and allow deeper networks.
    - Dropout in fully connected layers to reduce overfitting.
5. Use convolutional layers specialized for images
    - Stacked conv layers with small kernels (3×3) and pooling/strided conv to build deep feature extractors.
    - This exploits spatial structure of faces far better than a shallow fully-connected net.

Research on facial recognition shows CNNs with ReLU/Parametric ReLU and multiple layers significantly improve accuracy over shallow networks.

4. Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks

Scenario: Streaming service wants to recommend movies based on viewing history without labeled data.

RBMs are two-layer undirected graphical models (visible + hidden units, no intra-layer connections) that can model user–item interactions and learn latent preferences in an unsupervised .

Architecture and idea

1. Visible units
    - Represent movies (items).
    - For each user, visible units encode their ratings or views (e.g., binary: watched/not; or multinomial via softmax for rating levels).
2. Hidden units
    - Capture latent factors (genres, themes, user tastes) that explain patterns in viewing behavior.
    - Each hidden unit connects to all visible units; no hidden–hidden or visible–visible connections (restricted structure).
3. Shared parameters across users
    - One RBM model with shared weights between hidden units and each movie's visible unit.
    - Training averages updates over all users, so if two users rated the same movie, they contribute to the same weight updates.

Training (unsupervised)

- Use user–movie interaction matrix (e.g., Netflix-style ratings or watch/no-watch). No explicit labels needed beyond observed interactions.sfu+1
- Train via contrastive divergence to maximize likelihood of observed data.
- The RBM learns to reconstruct a user's observed pattern of movie interactions from the hidden representation.

Making recommendations

For a given user $u$and movie i:

1. Clamp the user's known ratings/views onto the visible units.
2. Infer hidden activations: compute $\hat{p_j} = p(h_j = 1 \mid V)$for all hidden units.
3. Predict rating/probability for movie i:

$$p(v_i = 1 \mid \hat{p}) = \frac{\exp\left(b_i + \sum_j \hat{p_j} W_{ij}\right)}{\sum_k \exp\left(b_i^{(k)} + \sum_j \hat{p}_j W_{ij}^{(k)}\right)}$$

(for multinomial ratings) or a simpler sigmoid for binary.

4. Use expected value as predicted rating $R(u, i)$; recommend movies with highest predicted scores that the user hasn't watched.

RBMs were successfully applied to the Netflix dataset, outperforming SVD-based collaborative filtering and forming part of top-performing ensemble methods.

5. Deep Learning Applications, Representation Learning

Scenario: Manufacturing company wants to detect defective products from sensor readings every second.

Proposed deep learning solution (representation learning + anomaly detection)

Use an unsupervised deep autoencoder (possibly recurrent or 1D CNN-based) to learn compact representations of normal sensor sequences, then detect defects via reconstruction error and/or latent-space anomalies.

Architecture overview

1. Input representation
    - Sensor data: multivariate time series (e.g., temperature, vibration, pressure, etc.) sampled every second.
    - Use sliding windows (e.g., 60–300 seconds) to form input sequences.
2. Model choice
    - Recurrent Deep Autoencoder (e.g., LSTM/GRU encoder–decoder) or 1D CNN autoencoder to handle temporal dependencies.
    - Encoder compresses each window into a low-dimensional latent vector capturing normal operating patterns.
    - Decoder reconstructs the original window from the latent code.

3. Training strategy
    - Train primarily on normal operation data (most of the data; defects are rare).
    - Loss: reconstruction error (e.g., MSE) between input and output sequences.
    - This is unsupervised / self-supervised representation learning; no defect labels required.

4. Defect / anomaly detection
   For each new sensor window:
    - Compute reconstruction error (e.g., per time step, per sensor, aggregated).
    - Optionally also use the latent vector with a density model (e.g., Gaussian Mixture Model) to detect points in latent space that are far from normal clusters.
    - Define thresholds (statistical or learned) on error/latent distance; exceedances → potential defect.

5. Integration into manufacturing workflow
    - Run model in near real-time on streaming sensor data.
    - When anomaly score exceeds threshold, trigger alerts, log the window, and optionally halt the line or flag the product for inspection.
    - Use historical anomalies to refine thresholds and possibly train a supervised classifier later if labeled defects become available.

This design leverages representation learning (latent embeddings of sensor behavior) and deep autoencoders for scalable, unsupervised defect detection in high-frequency sensor data

RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are wellstructured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand